



21 Lab 2 — Structured Outputs: From LLM Text to Python Objects

[Code](#)

Make the model emit a schema you can trust, then map it straight onto a Python class

21.1 Goal

In [Module 2](#) you saw that an LLM only ever produces *tokens*, and that for anything that must be exact — tool calls, structured JSON — you want a **low temperature** and a way to *force the shape* of the output. This lab is that idea made concrete.

Why JSON? When your program and an LLM need to exchange data, they need a shared “language” — a text format both sides understand. That language is almost always **JSON** ([see the JSON primer](#)). It is the lingua franca of working with LLMs for three reasons: every model has seen enormous amounts of it during training (so models are good at producing it), it is plain text (so it survives any network call), and — crucially — it maps cleanly onto the data structures you already use in Python. A JSON object becomes a Python **dict**, a JSON array becomes a **list**, and JSON strings/numbers/booleans become **str / int / float / bool**. Better still, as you will see below, a JSON object can be mapped *directly* onto a Python **class**, so the model’s reply arrives as a real object with typed attributes (`recipe.servings`) instead of a string you have to pick apart.

This back-and-forth — your code describes a shape, the model fills it in, you read it back as native Python — is **central to everything you will build in this course**. But there is a catch worth stating plainly: an LLM is **not 100% deterministic**. Even at `temperature=0` the model can occasionally return malformed JSON, refuse, miss a field, or get the type wrong, and the network underneath can time out or rate-limit you. Communication with an LLM is *probabilistic*, so you must design for **fault tolerance** — validate what comes back and retry when it is wrong, rather than assuming success. That is exactly what the retry loop in this lab teaches.

Finally, none of this is free. Every call you make has a **real-world cost** — money (you pay per token, input *and* output), time (a round trip is hundreds of milliseconds to several seconds), and compute (a GPU somewhere is running the model). Retries multiply that cost, larger schemas mean more output tokens, and a chatty system prompt is billed on every single call. Throughout the course, treat each LLM call as something you spend — be deliberate about how many you make and how big they are.

You will build a small program that:

1. Sends a blob of free-form text to the VT ARC LLM.
2. Asks it — using the OpenAI **structured outputs** feature — to return data that conforms to a schema you define.
3. Receives the result already mapped onto a typed **Python object**, not a string you have to hand-parse.
4. Wraps the call in `try/except` with **automatic retries**, because the network and the model both fail sometimes.
5. Prints the resulting object to the console.

Then you will do it again, mostly from scratch, on a new schema — guided only by a UML class diagram.

Why this is the natural sequel to Lab 1

Lab 1 gave the agent a *tool* and let it run code. The other half of “the model talks to your program” is the reverse direction: getting the model’s output into a shape your program can use **reliably**. Structured outputs is how every later lab will turn model responses into function arguments, database rows, and UI state. It is also a load-bearing skill for the mid-term project, where you must turn grant text and faculty data into structured records.

21.2 Before you start

You need a working VSCode-to-server connection and the `ads_5984` conda environment, both covered in [Provisioning Your Environment](#). You also need your **VT ARC API key** — the same one from the OpenCode setup in Provisioning (from <https://llm.arc.vt.edu/>). Reuse it if you already have one.

Important

You must be on the **eduroam** network or the **Cisco VPN** for your program to reach the VT ARC model endpoint.

21.3 Step 1 — Connect and make the project folder

1. Open VSCode and connect to your assigned server (`ads1` or `ads2`) via Remote-SSH (see [Provisioning](#)).
2. Open a terminal: **Terminal** → **New Terminal**. Confirm the prompt shows the `ads_5984` environment (run `conda activate ads_5984` if not).
3. Create and open the project folder:

```
mkdir -p ~/ads_5984/structured_outputs  
cd ~/ads_5984/structured_outputs
```

4. In VSCode: **File** → **Open Folder...**, choose `/home/<your-userid>/ads_5984/structured_outputs`, and reopen. All files below go in this folder.

21.4 Step 2 — Check your tools

This lab needs just two libraries beyond the standard library. Verify them in the VSCode terminal:

```
python -c "import openai, pydantic, dotenv; print('deps OK')"
```

If the import fails, install into the `ads_5984` environment:

```
pip install openai pydantic python-dotenv
```

21.5 Step 3 — Create the environment file

Create a file named `.env` in the project folder. It points at VT ARC's OpenAI-compatible API and the GLM-5.3 model.

```
OPEN_AI_ENDPOINT=https://llm-api.arc.vt.edu/api/v1
OPEN_AI_API_KEY=<your API key from https://llm.arc.vt.edu here>
OPEN_AI_MODEL=GLM-5.3
```

Replace the API-key line with your actual key and save.

Two things that will bite you

1. **Never commit** `.env`. It contains a secret. If you `git init` this folder, add `.env` to `.gitignore` first.
2. **Model identifier**. If your first run fails with a *model not found* / 404, the API expects the exact id `GLM-5.3` (hyphen, no space). Your instructor will confirm the exact id for the current term.

21.6 Step 4 — Define the shape you want: Python classes

The whole point of structured outputs is that *you* declare the shape, in Python, and the model has to fill it in. We use **Pydantic** models — plain Python classes that also know how to validate themselves.

New to Pydantic? Read this first

Our target is a recipe. A *Recipe* contains a list of *Ingredient*s — a simple composition you will see constantly.

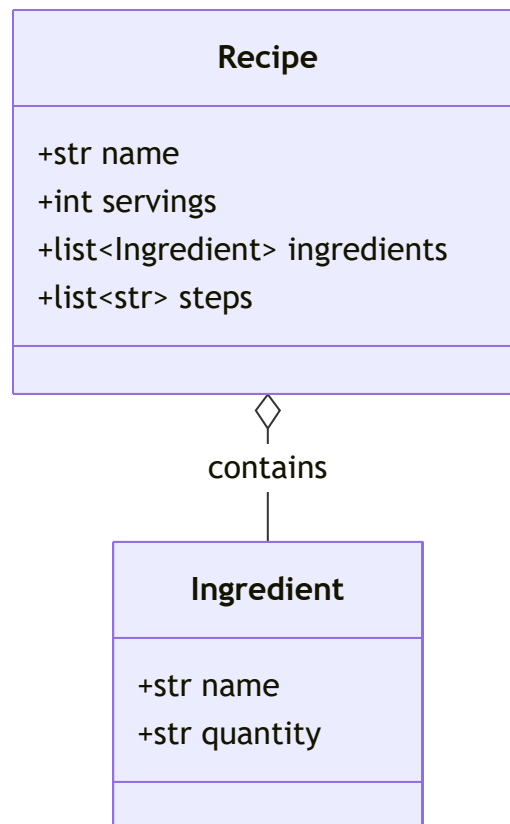


Figure 21.1: The target schema for the worked example. A *Recipe* is composed of *Ingredient* objects plus scalar and list fields. This class structure IS the schema the model is forced to honor.

21.7 Step 5 — Write the extractor

This is the heart of the lab: one file that defines the classes, calls the model with structured outputs, validates and retries, and returns a typed object. Create `extract.py` and paste the following. Read the comments — each block does a job named in [Module 2](#).

```
"""Lab 2 – Structured Outputs.

Ask the VT ARC LLM to read free-form text and return a strongly-typed Python
object, using the OpenAI structured-outputs feature
(response_format=<PydanticModel>), with validation and automatic retries.

There is no agent loop and no tools here. This is the other half of
"the model talks to your program": getting model output into a shape your
code can rely on.
"""

from __future__ import annotations

import os
import sys
import time
from typing import Type, TypeVar

from dotenv import load_dotenv
from openai import OpenAI
from pydantic import BaseModel, ValidationError

# Load OPEN_AI_ENDPOINT / OPEN_AI_API_KEY / OPEN_AI_MODEL from .env
load_dotenv()

client = OpenAI(
    base_url=os.environ["OPEN_AI_ENDPOINT"],
    api_key=os.environ["OPEN_AI_API_KEY"],
)
MODEL = os.environ["OPEN_AI_MODEL"]

# ---- 1. The target shape: plain Python classes (Pydantic models) ----
# These classes ARE the schema. The model is constrained to fill them in.
class Ingredient(BaseModel):
    name: str
    quantity: str # keep it human, e.g. "1 1/2 cups"

class Recipe(BaseModel):
    name: str
    servings: int
    ingredients: list[Ingredient]
    steps: list[str]
```

```
SYSTEM_PROMPT = (
    "You are a precise information-extraction service. Read the user's text and "
    "return ONLY data that matches the requested schema. Do not invent fields or "
    "facts; if a value is genuinely absent, make the most faithful inference the "
    "text supports."
)
```

```
# A TypeVar lets one function work for ANY Pydantic schema (Recipe, Movie, ...).
T = TypeVar("T", bound=BaseModel)
```

```
def _validate_lenient(schema: Type[T], text: str) -> T:
    """Validate `text` against `schema`, repairing one known corruption.

    Real endpoints have real bugs: the VT ARC server intermittently prepends
    a stray '{' line in json_schema mode, so the body arrives as
    '{\\n{\\n  "name": ...}' – invalid JSON. If plain validation fails, drop
    everything up to the SECOND '{' and validate again. This is exactly the
    fault-tolerance discipline this lab teaches: never assume the model (or
    the server) returned what you asked for.
    """
    try:
        return schema.model_validate_json(text)
    except ValidationError:
        first = text.find("{")
        second = text.find("{", first + 1)
        if second == -1:
            raise
        return schema.model_validate_json(text[second:])
```

```
def extract(text: str, schema: Type[T], *, max_retries: int = 3) -> T:
    """Extract `text` into an instance of `schema` using structured outputs.

    `response_format=schema` is the structured-outputs feature: the server is
    asked to constrain generation to the schema, so the reply already maps onto
    the class. temperature=0 because we want the SAME parse every run, not
    creativity (Module 2: low temperature for structured / JSON output).
```

The try/except gives us several layers of safety:

- API/network errors (timeouts, 5xx, rate limits) -> retry with backoff
- model refusals (it can decline instead of answering) -> retry
- a server that mangles strict-mode output -> non-strict fallback call
- validation errors (output that does not fit the schema) -> retry

```
"""
last_error: Exception | None = None
messages = [
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": text},
```

```

]

for attempt in range(1, max_retries + 1):
    try:
        try:
            completion = client.chat.completions.parse(
                model=MODEL,
                temperature=0,
                messages=messages,
                response_format=schema, # <-- the structured-outputs feature
            )
            message = completion.choices[0].message

            # The model may refuse rather than answer. Not a parse error.
            if getattr(message, "refusal", None):
                raise ValueError(f"model refused: {message.refusal}")

            parsed = message.parsed
            if parsed is None:
                # Fallback: if the server did not honor the schema feature,
                # validate the raw JSON text ourselves. Same result, by hand.
                parsed = _validate_lenient(schema, message.content or "")

        except ValidationError:
            # ENDPOINT QUIRK: .parse() requests the schema in *strict*
            # mode, and the VT ARC server frequently mangles strict-mode
            # output (a stray '{' before the JSON). The cure is the same
            # fault-tolerance idea this lab teaches: fall back to a plain
            # (non-strict) json_schema request and validate ourselves.
            resp = client.chat.completions.create(
                model=MODEL,
                temperature=0,
                messages=messages,
                response_format={
                    "type": "json_schema",
                    "json_schema": {"name": schema.__name__,
                                    "schema": schema.model_json_schema()},
                },
            )
            parsed = _validate_lenient(
                schema, resp.choices[0].message.content or ""
            )

        return parsed # success: a fully-typed Python object

    except (ValidationError, ValueError) as exc:
        last_error = exc
        print(f"[attempt {attempt}/{max_retries}] bad output: {exc}",
              file=sys.stderr)
    except Exception as exc: # network / API / rate-limit, etc.
        last_error = exc

```

```

print(f"[attempt {attempt}/{max_retries}] API error: "
      f"{type(exc).__name__}: {exc}", file=sys.stderr)

if attempt < max_retries:
    time.sleep(2 * attempt) # simple linear backoff

raise RuntimeError(
    f"failed to extract {schema.__name__} after {max_retries} attempts"
) from last_error

# ---- 2. Some messy, free-form text to extract from ----
RECIPE_TEXT = """\
Grandma's buttermilk pancakes feed about four people. You'll need 1 1/2 cups of
all-purpose flour, 3 1/2 teaspoons of baking powder, 1 teaspoon of salt, 1
tablespoon of sugar, 1 1/4 cups of milk, 1 large egg, and 3 tablespoons of
melted butter. Whisk the dry ingredients together, make a well, and pour in the
milk, egg, and butter. Stir until just smooth. Heat a greased griddle over
medium and pour the batter by the 1/4 cup. Flip when bubbles form on the
surface, then cook until golden brown.
"""

def main() -> None:
    recipe = extract(RECIPE_TEXT, Recipe)

    # `recipe` is a real Python object, NOT a string of JSON. Prove it.
    print("=== Parsed Recipe object ===")
    print(repr(recipe))
    print()
    print(f"name:          {recipe.name}")
    print(f"servings:      {recipe.servings} (type: {type(recipe.servings).__name__})")
    print(f"ingredients: {len(recipe.ingredients)}")
    for ing in recipe.ingredients:
        print(f"  - {ing.quantity} {ing.name}")
    print(f"steps:          {len(recipe.steps)}")
    for i, step in enumerate(recipe.steps, start=1):
        print(f"  {i}. {step}")

    # Because it is typed, you can compute on it directly – no string surgery.
    print("\n--- round-trip back to JSON ---")
    print(recipe.model_dump_json(indent=2))

if __name__ == "__main__":
    main()

```

The flow inside `extract()` — the call, the three failure modes, and the retry — is the part to internalize. Read it as a short conversation between your code, the model, and the schema.

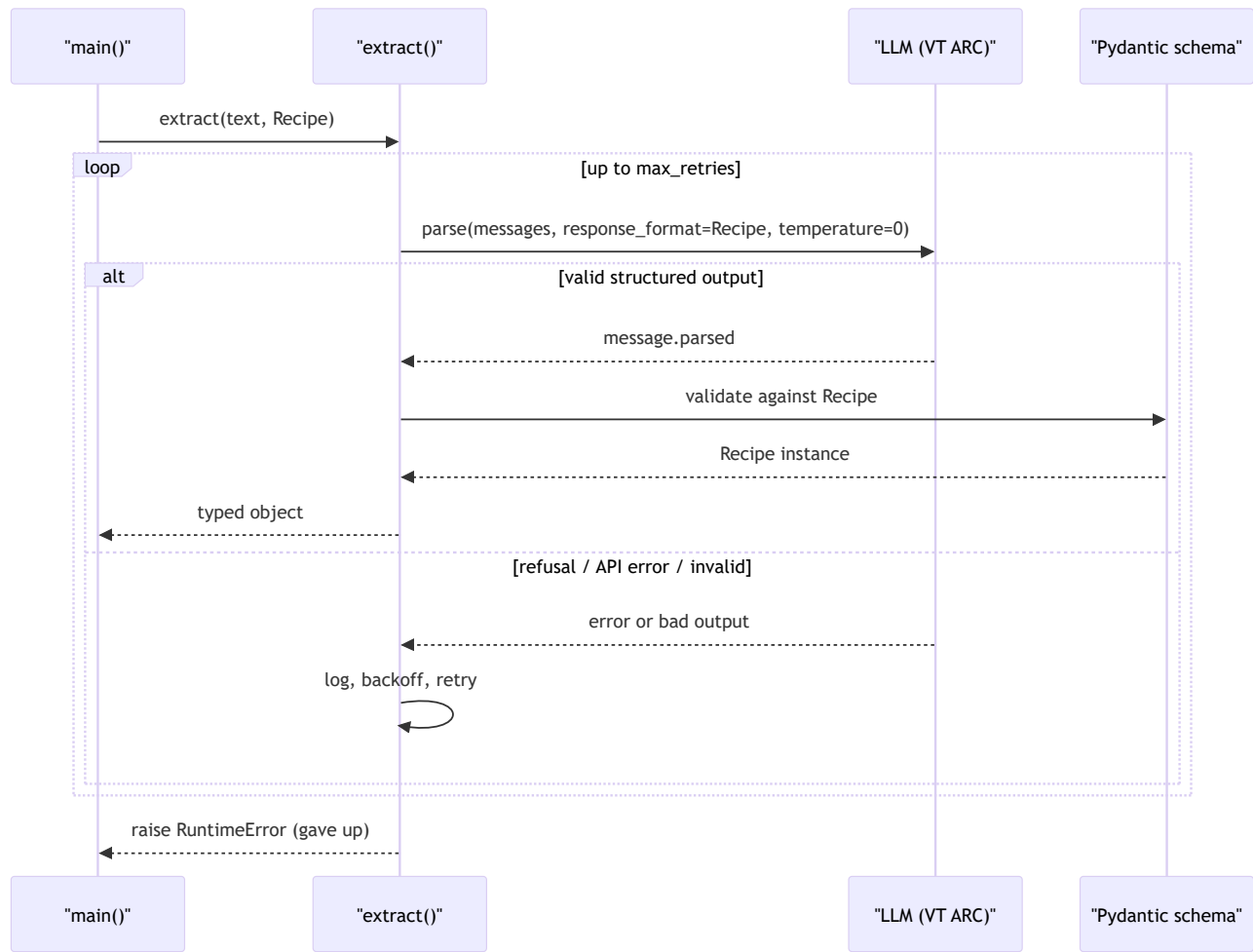


Figure 21.2: The extract-validate-retry flow. Each attempt either returns a validated, typed object or falls through to a logged retry with backoff; after max_retries the function raises.

21.8 Step 6 — Run it

From the VSCode terminal (with `ads_5984` active):

```
cd ~/ads_5984/structured_outputs
python extract.py
```

You should see the `Recipe` object printed, its fields populated from the prose — `servings` is a real `int`, `ingredients` is a list of `Ingredient` objects — and the same data round-tripped back to JSON. The model read the paragraph and handed your program a typed object.

21.9 What you just saw

Three things worth naming, each tying back to [Module 2](#):

1. **Structured outputs is *constraint*, not *hope*.** Passing `response_format=Recipe` asks the server to keep the generation inside your schema. Compare that to “please reply with JSON” in a plain prompt and then praying it parses. You are removing a whole class of failure rather than catching it after the fact.

2. **temperature = 0 is the correct setting here.** You want the *same* extraction every run. This is exactly the low-temperature, top-p-at-default regime the Module 2 sampling section recommended for structured/JSON output. Creativity is the enemy of a parser.
3. **Robust ≠ optimistic.** A real call can time out, get rate-limited, return a refusal, or (rarely) miss the schema. The `try/except` + retry loop is not boilerplate — it is the difference between a demo and something you can put in an agent that runs unattended.

Structured outputs vs. tool/function calling

They use the same underlying mechanism (the model is constrained to emit a schema). Rule of thumb: use **structured outputs** when you want the model's *answer* in a fixed shape (this lab). Use **function/tool calling** when you want the model to *request an action* your code performs (Lab 1's tool). Primer 07 covers both in depth.

21.10 Exercise — do it from scratch

Now build the same pattern for a different schema, with much less scaffolding. Make a **new file** `movies.py` in the same folder.

Your task: extract a `Movie` (with its cast) from a paragraph of free-form text, using structured outputs, validation, and retries, then print the typed object.

You are given the target schema as a UML class diagram. Translate it into Pydantic models yourself.

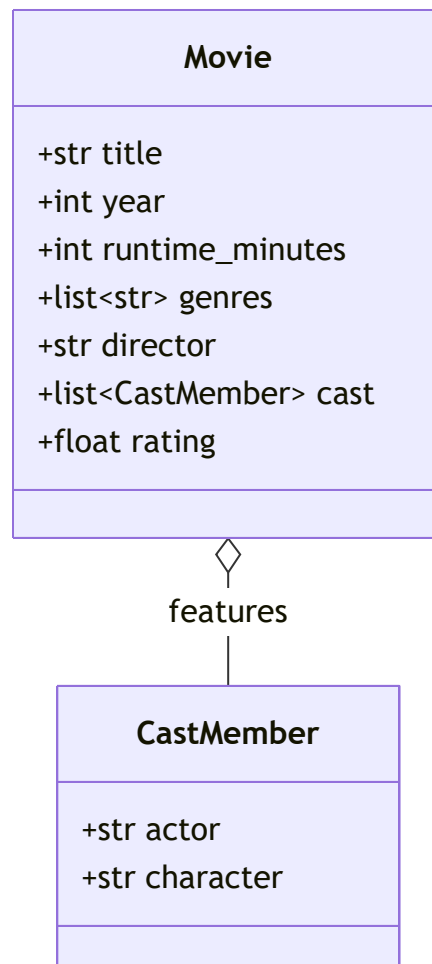


Figure 21.3: Target schema for the exercise. Translate this UML directly into Pydantic models: a `Movie` composed of `CastMember` objects, plus scalar and list fields.

Requirements:

1. Define `CastMember` and `Movie` as Pydantic models matching the diagram exactly (mind the types: `year` and `runtime_minutes` are `int`, `rating` is `float`, `genres` is a list of strings, `cast` is a list of `CastMember`).
2. Reuse the *pattern* from `extract.py`: a system prompt, `client.chat.completions.parse(..., response_format=Movie, temperature=0)`, a `try/except` that retries on API errors, refusals, and `ValidationError`, and the non-strict + `_validate_lenient` fallback path. (Easiest and best: `from extract import extract` and reuse the generic function — that reuse *is* the lesson.)
3. Write your own `MOVIE_TEXT` — a paragraph (in plain prose, *not* JSON) describing a film you like, including its title, year, runtime, genres, director, a few cast members with their characters, and a rating.
4. Print the resulting `Movie` object, then iterate over `movie.cast` printing `actor` → `character`.

Stretch goals (optional):

- Add an `Optional[str] tagline` field and a `box_office_millions: float | None` field. Notice the model leaves them `None` when the text does not mention them — and does *not* invent values, because your system prompt forbade it.
- Make `rating` validate to the range 0–10 (hint: Pydantic `field_validator` or `Annotated[float, Field(ge=0, le=10)]`). Feed it text with an out-of-range rating and watch the retry loop fire.
- Factor out the generic `extract(text, schema)` function so the *same* function handles both `Recipe` and `Movie`. (It already can — `extract.py` was written generically with a `TypeVar`. Confirm you understand why.)

Stuck? A nudge (not the answer)

21.11 Troubleshooting

Common errors and fixes

21.12 Key Takeaways

- **Structured outputs convert a probabilistic text generator into a typed data source you can program against.** — The whole lab turns “the model emits tokens” into “the model hands my code a `Recipe` object with a real `int` for servings.” That bridge — a Python class *is* the schema the model is constrained to fill — is the other direction of the same machinery Lab 1 used for tool calls, and it is the seam where free-form language meets reliable software.
- **Constraint beats hope, and the difference is a whole class of bugs you stop catching after the fact.** — `response_format=Recipe` asks the server to keep generation inside your schema, which is categorically stronger than “please reply with JSON” and praying it parses. Pair that with `temperature=0`, because for extraction you want the *same* parse every run — creativity is the enemy of a parser.
- **An LLM call is probabilistic and costly, so design for fault tolerance, not success.** — Even at `temperature=0` the model can refuse, miss a field, or mistype one, and the network can time out or rate-limit

you. The retry loop is not boilerplate; it is the discipline of never trusting model output blindly, and it is what separates a demo from something an unattended agent can depend on. Every retry, every extra field, every chatty system prompt is also spent in the three currencies — money and latency on each round trip — so make calls deliberately.

- **One generic function over a `TypeVar` is the reuse the lab is really teaching.** — `extract(text, schema: Type[T]) -> T` returns a `Recipe` in one call and a `Movie` in another with zero new parsing code. Pushing the variation into the schema and keeping the call site generic is a pattern you will reach for constantly once you have many shapes to extract.
- **This is the load-bearing skill that makes the mid-term project possible.** — The grant-teams tool lives or dies on turning messy grants.gov solicitation text and Elsevier/Scopus publication data into clean, typed records you can store, filter, and reason over ([project spec](#)). Every later lab — function arguments, database rows, UI state — is this same extract-validate-retry pattern wearing a different schema.

21.13 Self-check

1. In `extract.py`, which single argument turns an ordinary chat call into a *structured-outputs* call? What does it constrain?
2. Why is `temperature=0` the right choice for this task, in the vocabulary of the Module 2 sampling section?
3. Name the three distinct failure modes the `try/except` block handles, and why a refusal is *not* a `ValidationError`.
4. The `extract()` function takes `schema: Type[T]` and returns `T`. Explain how the *same* function returns a `Recipe` in one call and a `Movie` in another.
5. What is the difference between structured outputs and tool/function calling, and when would you reach for each?